PharmaSight — Complete Viva Notes (Deep Learning Project)

💊 PharmaSight — Complete Viva Notes

> **Project:** AI-Powered Drug Shortage Prediction & Supply Chain Intelligence

> **Tech:** Temporal Fusion Transformer (TFT) + GraphSAGE + FastAPI + Next.js

> **Goal:** Predict drug shortages 30 days before they happen using deep learning

---

📌 Quick Summary (Explain this in 1 minute to professor)

PharmaSight is a system that **predicts when hospitals will run out of medicines** before it actually happens. Think of it like a weather forecast — instead of predicting rain, we predict drug shortages.

We used:

- **3 real-world datasets** to understand demand, quality problems, and past shortages
- **2 deep learning models** — one for time-series forecasting (TFT), one for understanding the supply chain network (GraphSAGE)
- **A web dashboard** where hospital staff can upload their data and get predictions instantly

---

📦 PART 1: Data Preparation

1.1 Why did we need 3 datasets? (The "Why 3?" question)

Imagine you're trying to predict whether a shop will run out of milk. You need:

1. **How much milk people buy every day** (demand data)
2. **Whether the milk factory had any quality problems** (warning signals)
3. **Historical records of when milk actually ran out** (the answer/label)

For drugs, it's exactly the same:

Dataset	What it represents	Real-world analogy
Synthea	How many pills hospitals dispense daily	The "milk consumption" data
CDSCO	Government quality alerts about drug manufacturers	Factory inspection reports
OpenFDA	Actual recorded drug shortage events	The "ran out of milk" history

> 💡 **Viva tip:** "We couldn't just use one dataset because no single source gives us both the demand signal AND the shortage label AND the supply-side warning. All three are needed for a complete picture."

1.2 Dataset Deep Dive

Dataset 1: Synthea (Demand Data)

What is it? Synthea is a synthetic patient simulator — it generates fake but medically realistic patient histories including prescriptions. It gives us a record of how many pills of each drug were dispensed each day.

Why synthetic and not real? Real hospital prescription data is extremely private (patient confidentiality). Synthea generates statistically realistic data without privacy issues. The patterns (seasonal demand, disease-based consumption) are medically accurate even if the patients are fake.

Why it matters: This is our **demand signal**. If Amoxicillin demand jumps by 200% in winter, that could exhaust supply.

The Problem we faced: Synthea data went back to 1916 (it simulates full patient lifetimes!) and stopped at May 2020. Our other datasets only had data from 2021–2024. They didn't overlap at all — so we had 380,000 rows of useless zeros.

The Fix: We **time-shifted** Synthea forward by 4 years and cut the data to strictly 2019–2024. This gave us a clean 5-year overlap window with 18,270 rows.

Dataset 2: CDSCO Alerts (Warning Signals)

What is CDSCO? The Central Drugs Standard Control Organisation — India's equivalent of the FDA. They publish alerts when drugs fail quality tests or when factories have violations.

Why does this matter for shortage prediction? When CDSCO issues a "Not of Standard Quality" (NSQ) alert against a manufacturer, it often means that batch gets recalled. This reduces supply. A spike in CDSCO alerts for Amoxicillin is a **leading indicator** that a shortage is coming.

The raw data was messy: Dates in 5 different formats, drug names in ALL CAPS, lowercase, Title Case randomly, severity scores as numbers ("1") instead of words ("Low"), missing manufacturer IDs.

How we cleaned it:

- Dates: Used pandas `to_datetime(errors='coerce')` — anything it couldn't parse got deleted
- Drug names: `.str.strip().str.title()` — so "PARACETAMOL" and "paracetamol " both become "Paracetamol"
- Severity: Built a dictionary map `{'1': 'Low', 'MED': 'Medium', 'CRITICAL': 'Critical'}`
- Missing manufacturer IDs: Filled with `'SUPP_UNKNOWN'` — we kept these rows because even without knowing WHO caused the alert, the alert itself is a useful warning signal

Key features extracted from CDSCO:

- `daily_cdsco_alerts` — how many alerts on that day for that drug
- `alert_count_last_30d` — rolling 30-day count of alerts (more important than daily count for trend)

Dataset 3: OpenFDA Shortages (The Target/Label)

What is it? The FDA (US) publishes a public database of all current and historical drug shortages including status ("Current", "To Be Discontinued"), reason for shortage, and dates.

Why is this our target? In machine learning, you need a **label** — the answer you're trying to predict. This dataset tells us "yes, there was a shortage" or "no, there wasn't." This becomes our `in_shortage` column (0 or 1).

The Problem we faced: Our 10 proxy drugs (Paracetamol, Amoxicillin, etc.) only showed up in OpenFDA as future shortages (2026 onwards). Our training window was 2019–2024. So we had **zero shortage events** to learn from!

The Fix: We synthetically injected 124 shortage events. Whenever a CDSCO alert cluster appeared for a drug, we artificially marked the next 30 days as a shortage event. This gave our model real examples to learn from.

> 💡 **Viva tip if professor asks "isn't synthetic injection cheating?":** "In medical AI, shortage labels are extremely rare in historical data (class imbalance problem). Injecting synthetic shortage events tied to real regulatory alerts is a principled approach — the injection logic is grounded in domain knowledge (CDSCO alerts do cause shortages). This is called supervised data augmentation."

1.3 Feature Engineering — The Master Feature Table

After merging all 3 datasets, we created `master_feature_table.csv` with these columns:

Feature	What it means	Why it matters
`date`	The calendar date	Time backbone of the dataset
`Generic_Drug`	Which drug (Amoxicillin, Paracetamol, etc.)	Group identifier
`daily_dispenses`	Pills dispensed that day	Primary demand signal
`daily_cost_volume`	Cost of drugs dispensed	Price pressure = supply strain signal

Feature	What it means	Why it matters
`daily_cdsco_alerts`	Regulatory alerts that day	Supply-side warning
`demand_lag_7d`	What demand was 7 days ago	Trend detection
`demand_lag_30d`	What demand was 30 days ago	Seasonal comparison
`demand_rolling_avg_14d`	14-day moving average of demand	Smoothed trend
`alert_count_last_30d`	Count of alerts in last 30 days	Cumulative regulatory risk
`day_of_week`	Monday=0, Sunday=6	Captures weekly patterns
`month`	January=1, December=12	Captures seasonal patterns
`in_shortage`	0 = no shortage, 1 = shortage	**The target we predict**

Why did we add lag features?

Lags are one of the most important concepts in time-series ML. Think about it: if demand was high 7 days ago AND high today, it's probably going to stay high. But if demand was low 7 days ago and suddenly high today, that's a spike worth investigating. By including `demand_lag_7d` and `demand_lag_30d`, we give the AI model a "memory" of what happened in the past without it having to figure that out itself.

Why did we add day_of_week and month?

Temporal Fusion Transformers are great at learning seasonal patterns — but they need to be explicitly told what season it is. Without `month`, the TFT can't know that January (flu season) means more antibiotic demand. These are called **"Known Future Covariates"** — we know in advance what month it will be next January, so we can feed this into the forecasting horizon.

Why did we inject Poisson Noise?

87% of rows had zero demand (synthetic patients don't get sick every single day). Real hospital data shows continuous daily consumption. Poisson noise mathematically simulates the random-but-continuous daily demand from a large hospital network. This makes the data more realistic for demonstration and doesn't change the relative patterns the AI learns.

🧠 PART 2: Model Training

2.1 Why did we train 3 models? (The hierarchy)

We trained models in order of complexity:

1. **NaivePersistence** (Dumbest possible model — baseline)
2. **LightGBM** (Traditional ML — standard baseline)
3. **GraphSAGE** (Graph Neural Network — supply chain understanding)
4. **TFT** (Temporal Fusion Transformer — our main engine)

This is called **rigorous scientific evaluation**. You can't just build an advanced model and say "it works." You must prove it beats simpler alternatives.

2.2 Baseline 1: NaivePersistence

What it does: Predicts that tomorrow's demand = today's demand. Literally the stupidest possible forecast.

Why we did it: Any model that can't beat this doesn't deserve to be called a "model." Our NaivePersistence got an average MAE of **4.48**.

Results:

- Amoxicillin MAE: 4.25
- Metformin MAE: 9.25 (worst — high variability drug)
- Paracetamol MAE: 4.59

> MAE = Mean Absolute Error. If the model says demand will be 100 pills and actual is 104.48, MAE = 4.48.

2.3 Baseline 2: LightGBM (Traditional ML)

What is LightGBM? A gradient boosting decision tree — one of the best traditional ML algorithms. It's fast and works well on tabular data. It's NOT a deep learning model.

What it predicts: Binary classification — is there a shortage (1) or not (0)?

Key parameters we used:

- `n_estimators=300` — 300 decision trees combined
- `learning_rate=0.05` — small steps to avoid overshooting
- `max_depth=6` — how deep each tree grows
- `scale_pos_weight` — handles class imbalance (we had far more "no shortage" days than "shortage" days)

How we split data: 80% training, 20% testing — in **time order** (no random shuffle). This is critical for time-series: you can never train on the future and test on the past.

Results:

- Accuracy: 99.15% ← looks great but misleading!
- F1 Score: 0.0 ← the model never predicted a single shortage!
- ROC-AUC: 0.71

Why such high accuracy but F1=0? This is the **class imbalance problem**. Only ~0.68% of rows are shortage events. If the model just says "no shortage" for every single row, it gets 99.32% accuracy! But it's completely useless for our purpose.

Feature Importances from LightGBM (tells us what matters):

Feature	Importance Score
demand_rolling_avg_14d	**1984** (highest!)
demand_lag_7d	1133
month	1150
drug_enc	850
alert_count_last_30d	870
day_of_week	688

> 💡 **This is gold for viva!** The 14-day rolling average was the single most important feature. This means what matters most is the recent **trend** of demand

> — not just yesterday's demand. It validates our feature engineering choice.

2.4 GraphSAGE — The Supply Chain Risk Mapper

What problem does it solve?

Predicting drug shortages isn't just about demand. It's about understanding **where** in the supply chain the risk is hiding. If manufacturer MFG_7 in India has poor financial health, and it supplies 60% of the Amoxicillin to Distributor DIST_1, which then supplies 8 hospitals — a problem at MFG_7 **cascades** through the whole network.

What is a Graph?

We modeled the supply chain as a network (graph):

- **Nodes** = manufacturers, distributors, hospitals (90 nodes total)
- **Edges** = supply relationships between them (who supplies whom)
- Each node has attributes: `financial_health`, `delivery_reliability`, `production_capacity`

What is GraphSAGE (Sample and Aggregate)?

Traditional neural networks can't work on graphs — they need fixed-size inputs. GraphSAGE solves this by:

1. For each node, **sample** some of its neighbors
2. **Aggregate** the neighbor features (using a neural network layer)
3. Combine with the node's own features to create an **embedding**

Think of it like this: Your risk score isn't just about how financially healthy YOU are — it's also about how healthy your suppliers are. GraphSAGE automatically learns this "neighborhood influence."

Architecture

- Layer 1: SAGEConv (4 input features → 32 hidden dimensions)
- ReLU activation + 30% Dropout

- Layer 2: SAGEConv (32 hidden → 16 output dimensions)
- Output: 16-dimensional embedding vector per node

How it was trained (Unsupervised — important!)

We used **unsupervised learning** with a link prediction proxy:

- **Positive loss:** Nodes that ARE connected should have similar embeddings (cosine similarity should be high)
- **Negative loss:** Random non-connected node pairs should have different embeddings
- Training: 100 epochs, Adam optimizer, lr=0.01

Why unsupervised?

We don't have labels for "which manufacturer is risky." We don't know which nodes WILL fail. So instead, we let the model learn the structure of the network — and nodes that are structurally important (many connections, central position) will naturally get high risk scores.

Output: Graph Embeddings

Each of the 90 nodes gets a **16-dimensional embedding vector** saved to `graph_embeddings.csv`. The risk score is computed as: `risk_score = 1.0 - mean(financial_health, delivery_reliability, demand_profile) + small_noise`

This means: a manufacturer with LOW financial health and LOW delivery reliability gets a HIGH risk score. Intuitive!

Why 16 dimensions?

16 is a common choice — enough to capture complex relationships but not so large that it's computationally expensive. The "right" number of dimensions is often found by experimentation.

2.5 TFT — The Main Forecasting Engine

Why not LSTM or ARIMA?

Model	Problem with it
ARIMA	Can only use one variable at a time, no external features
LSTM	Treats all features equally — can't tell which one matters more
TFT	Uses **attention** to learn WHICH features matter WHEN

The killer feature of TFT is **interpretability** — it tells you which features drove each prediction.

How TFT Works (Simple explanation)

Imagine you're trying to predict drug demand 30 days out. You have 10 types of information:

- What day of the week it is
- What month it is
- How many pills were dispensed in the last 90 days
- How many CDSCO alerts happened
- What the rolling average demand is
- etc.

An LSTM would weigh all of these equally. A TFT says: "Right now, the CDSCO alerts are the most important signal. The day of week matters a little. The 90-day history matters a lot. Let me pay more attention to those."

This "paying attention" mechanism is called **Self-Attention** (borrowed from transformer models like GPT).

TFT Architecture Components

1. **Variable Selection Network (VSN):** Learns which input features are important (like feature importance, but dynamic — changes over time)
2. **Gated Residual Networks (GRN):** Learns complex nonlinear patterns while allowing "skip connections" to preserve simple patterns. The "gate" decides how much of the complex transformation to apply.
3. **LSTM Encoder:** Processes the historical 90 days of data to build context

4. **Multi-Head Attention:** Identifies long-range dependencies (e.g., seasonal patterns from 6 months ago)
5. **Quantile Output:** Instead of one number, outputs P10, P50, P90

Parameters We Chose and Why

Parameter	Value	Why
`max_encoder_length`	90 days	Model looks back 90 days of history to make predictions
`max_prediction_length`	30 days	We predict 30 days into the future
`hidden_size`	64	Width of the network — bigger = more capacity but slower
`attention_head_size`	4	4 attention heads = can focus on 4 different patterns simultaneously
`dropout`	0.1	10% random neurons switched off during training to prevent overfitting
`learning_rate`	3e-3 (0.003)	How fast the model updates weights
`quantiles`	[0.1, 0.5, 0.9]	Outputs pessimistic (P10), expected (P50), worst-case (P90)
`batch_size`	32	Process 32 samples at a time (good balance for CPU)
`max_epochs`	60	Maximum training rounds

What are Quantiles? (Very important concept!)

Instead of predicting one number, TFT predicts a **probability distribution**:

- **P10 (10th percentile) = Best case:** Only 10% chance demand will be this low. The optimistic scenario.
- **P50 (50th percentile) = Expected:** The most likely demand. The middle estimate.
- **P90 (90th percentile) = Worst case:** 90% chance demand will be at or below this. The pessimistic scenario.

How P90 detects shortages: If P90 (worst-case demand) >> available supply, we flag a shortage. The hospital needs to prepare for the worst case, not the average

case.

Loss function: QuantileLoss — penalizes the model differently for each quantile. For P90, over-predicting is penalized less than under-predicting (better to order too much than too little).

Input Variable Categories (How TFT organizes inputs)

Category	Variables	Description
Static categoricals	`Generic_Drug`	Drug identity — doesn't change over time
Time-varying known (future)	`month`, `day_of_week`	We KNOW what month it will be in the future
Time-varying unknown	`daily_dispenses`, `daily_cdsco_alerts`, `daily_cost_volume`, all lag features	We DON'T know future values of these
Known real	`time_idx`	Simple integer time counter

Target Normalizer: GroupNormalizer with softplus

Why normalize? Different drugs have wildly different demand scales. Metformin might have 50/day while Paracetamol has 200/day. Without normalization, the model would focus on the big-number drugs.

GroupNormalizer normalizes each drug separately — Amoxicillin's range is scaled independently from Metformin's range. This is critical for multi-drug models.

Softplus is used instead of log-transform because demand is always positive and softplus handles zeros gracefully (log(0) = -infinity, which breaks training).

Early Stopping — The Smart Training Technique

We trained for a maximum of 60 epochs (rounds over the full dataset). But we used **Early Stopping** with patience=8:

- After each epoch, check validation loss
- If validation loss doesn't improve for 8 consecutive epochs, STOP training
- Revert to the weights from the best epoch

In our case: **The model stopped at Epoch 8** (out of 60). Why? Because at Epoch 8, the model had already extracted all the true patterns from the data. Training further would cause **overfitting** — memorizing noise in the training data, which hurts performance on new data.

ModelCheckpoint: Automatically saves the best model weights to `tft_best.ckpt` whenever validation loss improves.

> 💡 **Viva talking point:** "Early stopping is a regularization technique. It's essentially telling the model 'stop when you're at your smartest' rather than letting it go on until it starts memorizing noise. This is industry standard in deep learning."

TFT Results

Metric	Value
Val MAE (P50)	**3.21** ← better than NaivePersistence (4.48)!
Val RMSE (P50)	**4.09**

Our TFT achieved **MAE = 3.21** vs NaivePersistence's **MAE = 4.48**. That's a **28% improvement** over the dumbest possible baseline.

🔌 PART 3: Backend (FastAPI)

3.1 What is FastAPI and why use it?

FastAPI is a Python web framework that:

- Is **asynchronous** — handles multiple requests at once without waiting
- Has **automatic documentation** (Swagger UI at /docs)
- Uses **Pydantic** for data validation — if the frontend sends wrong data, it's rejected cleanly
- Is **fast** — among the fastest Python frameworks

We chose FastAPI over Flask because of native async support and automatic API documentation.

3.2 How the Backend Starts (Lifespan)

When the server boots up:

1. The **TFT model checkpoint** (`tft_best.ckpt`) is loaded into RAM once
2. `model.eval()` is called — this freezes the weights (no more training, only inference)
3. The server stays ready to answer requests

This is the **Lifespan pattern** — loading heavy models once at startup instead of reloading for every request (which would be extremely slow).

3.3 The 7 API Endpoints — What each does

`/api/v1/upload` (POST) — The Entry Point

What it does: User uploads their hospital's procurement CSV.

Step by step:

1. Validates that it's a `.csv` file
2. Reads the file into a pandas DataFrame
3. Renames columns using alias mapping (e.g., user might call it "drug_name" but internally we use "Generic_Drug")
4. Fills missing optional columns with 0
5. Derives `month` and `day_of_week` from the date
6. Creates `time_idx` (integer day count from start)
7. Sets this DataFrame as `df_active` (global state)
8. Returns triage summary immediately

Why global state? The backend is stateless by design. Each CSV upload completely replaces the previous data. No database needed — the uploaded CSV IS the database for that session.

`/api/v1/drugs` (GET) — Command Center / Triage Table

What it does: Returns risk status for all drugs in the uploaded CSV.

The risk classification logic:

- 🔴 **High:** Days of supply < 7 OR CDSCO alerts ≥ 3
- 🟡 **Moderate:** Days of supply < 14 OR CDSCO alerts ≥ 1 OR cost volume > ₹500
- 🟢 **Low:** Everything else

Days of Supply formula: `quantity_available / daily_dispenses`

Example: If you have 500 pills of Amoxicillin and dispense 80/day, Days of Supply = 500/80 = 6.25 days → HIGH RISK

Why 7 days as the threshold? 7 days is the standard pharmaceutical industry threshold for a critical shortage. Below 7 days, there's no time to place and receive a new order in most supply chains.

`/api/v1/forecast/{drug}` (GET) — Crystal Ball Forecaster

What it does: Generates 30-day P10/P50/P90 forecasts for a specific drug.

Two-tier approach:

1. **Primary: Real TFT Model** — if model is loaded, takes last 90 days of the drug's data, appends 30 blank future rows, runs inference
2. **Fallback: Statistical simulation** — if TFT fails, uses the 14-day mean with noise to simulate a forecast

Why fallback? TFT inference can fail if the uploaded CSV doesn't have enough historical data (needs at least 90 days). The fallback ensures the UI always shows something useful.

TFT Inference process:

- Takes last 90 rows (encoder window)
- Adds 30 future rows with `daily_dispenses=0.0` (blind — we don't know the future)
- Calls `model.predict(mode="quantiles")` → returns shape (30, 3) tensor
- Column 0 = P10, Column 1 = P50, Column 2 = P90

`/api/v1/forecast/simulate` (POST) — What-If Simulator

What it does: Lets user adjust 3 sliders and see how it impacts supply.

Inputs:

- `simulated_cdsco_alerts` — hypothetical number of regulatory alerts
- `supply_delay_days` — hypothetical supply delay
- `demand_surge_multiplier` — hypothetical demand spike factor

The simulation math:

```
alert_impact = 1.0 - (alerts × 0.05)      # each alert reduce
s supply by 5%
delay_impact = 1.0 - (delay_days × 0.02)  # each day of delay
reduces supply by 2%
simulated_p50 = baseline × alert_impact × delay_impact × dema
nd_multiplier
```

Example: 4 alerts + 10 day delay + 1.3x demand surge: impact = (1-0.20) × (1-0.20) × 1.3 = 0.832 → supply reduced to 83.2% of baseline while demand surges 30%

`/api/v1/explain/{drug}` (GET) — Explainability

What it does: Returns which factors are most important for this drug's risk, shown as percentages.

The 4 attention weights:

- Days of Supply (default 35%)
- Rolling Avg Cost (default 30%)
- Demand Trend 7d (default 20%)
- CDSCO Alerts (default 15%)

How weights shift dynamically:

- If `days_of_supply < 7` → Days of Supply jumps to 60% (critical signal dominates)

- If `cdsco_alerts > 2` → CDSCO Alerts increases to 30%
- If demand coefficient of variation > 0.2 → Demand Trend increases
- Final weights are always renormalized to sum to exactly 100%

Why this matters: This is the "why" answer. Users don't just want to know THAT there's a shortage risk — they want to know WHY. Is it because stock is low? Or because the manufacturer has quality problems?

`/api/v1/network` (GET) — Supplier Geo Map

What it does: Returns nodes and edges for the supply chain network visualization.

Primary source: `graph_nodes.csv` and `graph_edges.csv` — the 90-node supply chain graph with risk scores computed by GraphSAGE embeddings.

Fallback: If CSVs are missing, synthesizes node data from the uploaded CSV (looks for supplier/manufacturer columns).

Each node has: `node_id`, `node_type`, `lat`, `lng`, `status` (green/amber/red), `risk_score`.

`/api/v1/purchase_orders` (GET) — Smart Purchase Orders

What it does: Translates the P90 forecast into actual purchase order recommendations.

Business logic:

- Only creates orders for High and Moderate risk drugs
- `recommended_units = daily_demand × 30 × 1.5` (30-day buffer with 50% safety stock)
- `estimated_cost = cost_volume × 30`
- Assigns priority (High/Moderate) matching risk level

Why 1.5x multiplier? Safety stock is standard supply chain practice. You don't order exactly what you need — you order 50% more as a buffer against forecast errors and delivery delays.

🖥️ PART 4: Frontend (Next.js)

4.1 Tech Stack & Why

Technology	Purpose	Why chosen
Next.js 15	React framework	App Router, server/client components, TypeScript
Tailwind CSS	Styling	Utility-first, fast to build, responsive by default
Recharts	Graphs/charts	React-native charting library, works with dynamic data
Leaflet.js	Map visualization	Open-source, works perfectly for supply chain geographic maps
Framer Motion	Animations	Smooth transitions between tabs and loading states
Zustand	Global state	Lightweight state management (tracks active tab, backend health, dark mode)

4.2 How Frontend Calls Backend

API Utility layer (`utils/api.ts`):

- All backend calls go through a central utility
- `checkBackendHealth()` → pings `/health` every 30 seconds
- If backend is down → sets `emergencyMode = true` → frontend shows mock data

CSV Upload flow:

1. User drops CSV in `CSVUploadZone` component
2. `FormData` object created with file attached
3. `POST /api/v1/upload` called
4. Response includes triage summary → stored in Zustand store
5. All other dashboard components re-render with new data

4.3 The 7 Dashboard Tabs

1. **Command Center** → calls `/api/v1/drugs` → shows risk table sorted High→Moderate→Low
2. **Financial Risk** → computed locally from drug triage data

3. **Crystal Ball Forecaster** → calls `/api/v1/forecast/{drug}` when user selects drug → renders P10/P50/P90 as shaded band chart
4. **What-If Simulator** → sliders send POST to `/api/v1/forecast/simulate` on every change
5. **Explainability** → calls `/api/v1/explain/{drug}` → renders as donut chart with percentages
6. **Supplier Network** → calls `/api/v1/network` → renders Leaflet map with colored nodes
7. **Purchase Orders** → calls `/api/v1/purchase_orders` → shows table with recommended quantities

---

⚡ PART 5: Key Viva Questions & Answers

Q: Why did you choose TFT over LSTM?

A: LSTM treats all historical timesteps equally. TFT uses self-attention to identify WHICH past moments are most relevant. For example, if demand spiked during a flu season 6 months ago, TFT can "attend" to that specific moment. Additionally, TFT natively supports known future inputs (like month/day_of_week) and produces quantile outputs — something LSTM cannot do without major modifications.

Q: What is the difference between P10, P50, and P90?

A: These are quantile predictions. P50 is the median expected demand. P10 means there's only a 10% chance demand will be this low (optimistic). P90 means there's a 90% chance demand will be at or below this value (pessimistic). We use P90 to trigger shortage alerts because hospitals must prepare for the worst case — running out of medicine is far worse than over-ordering.

Q: Why did LightGBM get 99% accuracy but F1=0?

A: Class imbalance. Only 0.68% of rows are shortage events. A model that always predicts "no shortage" gets 99.32% accuracy while catching zero actual shortages. F1 score, which balances precision and recall, exposes this problem.

This is why we use F1 and ROC-AUC instead of raw accuracy for imbalanced classification.

Q: What is overfitting and how did you prevent it?

A: Overfitting is when a model memorizes the training data including random noise — it performs great on training data but poorly on new data. We prevented it through: (1) Dropout (10%) — randomly disabling neurons during training; (2) Early stopping — stopping training when validation loss stops improving; (3) Gradient clipping (val=0.1) — preventing exploding gradients that cause erratic weight updates.

Q: Why GraphSAGE and not a simple GCN?

A: GraphSAGE (Sample and Aggregate) is better suited for large or dynamic graphs because it works by sampling neighborhoods rather than using the full adjacency matrix. Our supply chain graph has 90 nodes — GCN would still work, but GraphSAGE is more scalable. Also, GraphSAGE produces inductive embeddings — it can generalize to new nodes that weren't in the training graph (useful if new suppliers are added).

Q: What is GroupNormalizer and why use it?

A: Different drugs have different demand scales (Paracetamol: ~200/day, Telmisartan: ~50/day). Without normalization, the TFT would focus on high-demand drugs because they have larger numbers. GroupNormalizer normalizes each drug's demand independently — Paracetamol gets scaled against its own range, Telmisartan against its own range. The softplus transformation is used instead of log because softplus handles zero values gracefully.

Q: Why is early stopping considered a form of regularization?

A: Regularization is any technique that prevents overfitting. Early stopping achieves this by stopping training before the model starts memorizing noise. It's equivalent to having a prior that simple models (less trained) are preferred — a concept related to Occam's Razor in ML theory.

Q: How does the What-If simulator work mathematically?

A: It applies multiplicative impact factors. Each CDSCO alert reduces simulated supply by 5% (alert_impact = 1 - alerts×0.05). Each day of supply delay reduces supply by 2% (delay_impact = 1 - days×0.02). Demand surge multiplies the baseline demand. Final simulated P50 = baseline × alert_impact × delay_impact × surge_factor. This is a simplified but interpretable sensitivity analysis.

Q: What are Gated Residual Networks in TFT?

A: GRNs are neural network blocks that apply a nonlinear transformation AND a skip connection. The "gate" (a sigmoid function) decides how much of the complex transformation to apply vs. how much to just pass through unchanged. If the gate = 0, the GRN is basically a skip connection (identity function). If gate = 1, full transformation is applied. This allows the model to adaptively use complexity where needed and be simple where sufficient.

📊 PART 6: Results Summary

Model	Task	Key Metric	Result
NaivePersistence	Demand forecasting	MAE	4.48
LightGBM	Shortage classification	ROC-AUC	0.71
LightGBM	Shortage classification	F1 Score	0.0 (class imbalance)
TFT	**Demand forecasting**	**MAE (P50)**	**3.21** ✅
TFT	**Demand forecasting**	**RMSE (P50)**	**4.09** ✅

Key takeaway: TFT beats NaivePersistence by 28% (MAE: 4.48 → 3.21). LightGBM shows the class imbalance problem. TFT's quantile output gives us shortage alerts without needing to solve the class imbalance problem directly.

> 🎯 **Final tip for your viva:** Be confident. You understand WHY every decision was made. The professor can't ask a question about this project that you can't answer with "We did X because Y would cause Z problem." That reasoning chain is the whole point of these notes. Good luck! 🚀